

تعريف بسيط

بإختصار مفيد كدا، بيقولك إن ال Scalability هو وصف لقدرة النظام على التعامل مع زيادة الحمل يعني زي ما اتكلمنا المرة الي فاتت على ال Reliability وإن النظام يكون Reliable لو كان شغال تحت اي ظرف، فاحنا هندمج التعريف دا مع انه يكون Reliable تحت ضغط

اي هو ال Load ؟

طب Load دا ممكن يتوصف ازاى؟

هو في الأول وفي الآخر الحاجات دي بتتحدد على حسب ال Bottleneck بتاع السيستم بتاعك بس احنا ممكن نوصف ال Load بأرقام ممكن نطلق عليها load parameters والحاجات دي تعتمد على بنية السيستم بتاعك زي ما قولنا فمثلا:

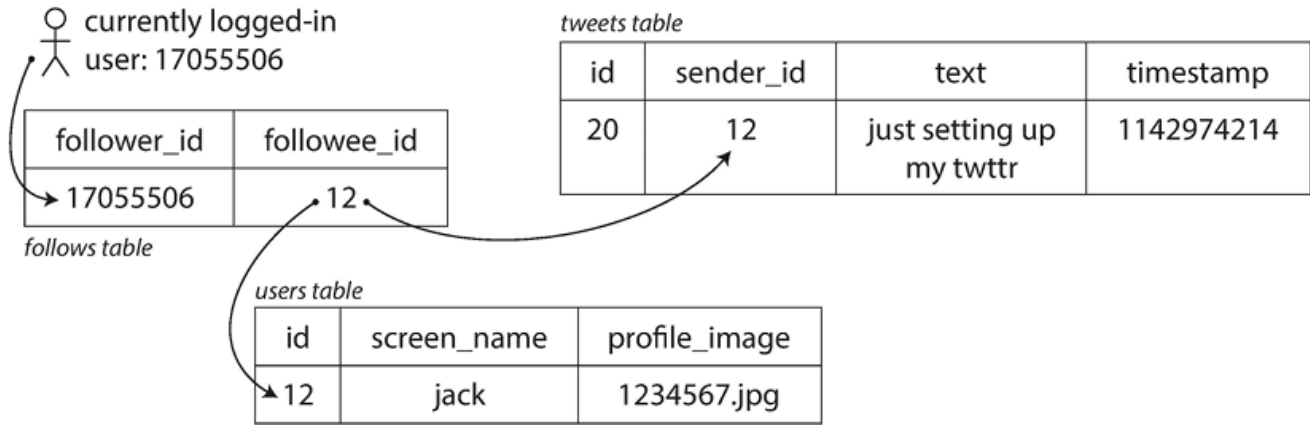
- عدد الريبكوستس الي بتوصل للسيرفر فالثانية الواحدة
- نسبة ال Read , Write الي بتكون في الداتايز
- عدد اليوزرز الأونلاين في نفس الوقت في شات مثلا
- او ال hit rat بتاع ال cache وهكذا...

المشكلة الي وقعت فيها تويتر! :

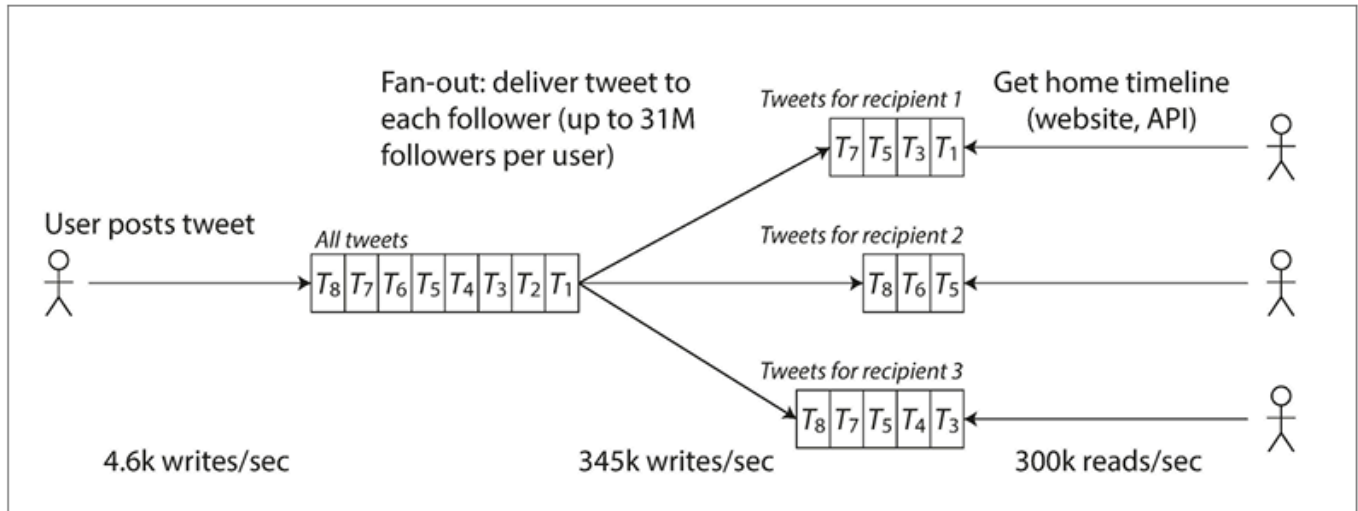
- تويتر حدد اللود وقتها وهو ان المستخدم يقدر ينشر رسالة جديدة لمتابعيه في متوسط 4.6 الف ريكوست في الثانية الواحدة، وأكثر من 12 الف ريكوست دا فحالة البيك (أعلى حاجة)، وكمان عدد التويتس الي اليوزر يقدر يشوفها حوالي 300 الف ريكوست في الثانية دا في المتوسط يعني
- لو جينا نقدرها ف انك تهندل ال 12 الف عملية كتابة في الثانية هي سهلة نوعا ما، لكن لو جينا لفكرة التوسع، كل يوزر ليه متابعين بيتابعهم وكمان كل يوزر عنده متابعين بيتابعوه ف في طريقتين للهندلة

1. في حاجة اسمها (Global Collection of Tweets) دا هيكون عبارة عن جزء من الداتا ييز بتاعتهم، فكل ما اليوزر بيعمل تويته جديدة بيتعملها انسيرت طبيعي في الحاجة دي، ف بالتالي انا لما يطلب اليوزر صفحته الرئيسية، انا هعرضله بوستات الي هو عاملهم فولو وادمجها بشكل مرتب على حسب الوقت (كويري بسيطة زي كدا)

```
select tweets.*, users.* from tweets
join users on tweets.sender_id = users.id
join follows on follows.followee_id = users.id
Where follows.follower_id = current_user
```



2. فكروا انهم يعملوا حفظ للكاش لكل تايم لاين خاصة باليوزرز، حاجة شبه كدا ال mailbox بحيث اني لو عملت تويته جديدة، هتروح تتسيف في الكاش الخاصة بـ التايم لاين بتاعت الفولورز بتوعي وأول ما يخشوا على التايم لاين هيقولوا احدث تويته نزلتها موجودة



طب الحاجة دي تتحل لو واحد زي حلاتي وحلاتك كدا، ناس على قد حالها وعندنا بالكثير لو شبت 1000 فولورز (مش مكملين ال 300)

فهتظهر مشكلة محمد صلاح (زي ما سماها بشمهندس أحمد الإمام)

وهي إني لو هطبق الطريقة الثانية، هضطر ابعت لكل متابعين الشخص المشهور دا الي قد يقدر عددهم بالملايين واعمل تحديث للكاش بتاعتهم ودا صعب انه يحصل

فالحل الي توصلوا ليه هيكون mix ما بين الحالتين 1 و 2

لو الشخص عادي زي وزبي هنختار الحالة الثانية مثلا

لو الشخص مشهور هندمج الاثنين

يا حبيبي يا Performance

الكلمة الي بيصدعك بيها اي حد معندوش بتلاتة تعريفة معلومة (أصل البيرفورمنس يا بشمهندس اصل ال..)

الكتاب قدر يوصف البيرفورمنس هنا ازاي؟

بص احنا محتاجين نثبت لود معين ممكن نستخدمه، فالبيرفورمنس هنا اصلها اي؟

- لو انا زودت اللود بس الريسورسيز الخاصة بالسيستم بتاعك (CPU, Memory, network bandwidth,...) بدون تغيير، أداء السيستم بتاعك هيتأثر ازاي؟
- طب يعم لو انا زودت اللود، هتحتاج قد اي علشان تزود الريسورسيز لو انت عايز تحافظ على الاداء بدون تغيير؟

لو جينا للمنطق، هل انت منطقًا يعني، تقدر تحسب دا من غير أرقام؟ ج- قطعًا لا!

فطبيعي هتحتاج ارقام

مثلا عندك زي Hadoop، ودا Batch processing system مشهور يعني، ال bottleneck بتاعه هو ال Throughput، وهو عبارة عن عدد الريكورد الي نقدر نعالجها في الثانية الواحدة أو الوقت الاجمالي الي بياخده الشغل على داتا سيز بحجم معين في الاونلاين سيستمز او ال response time دا بالنسبة للويب ابلكيشن

$$\text{response time} = \text{network time} + \text{queueing time} + \text{service time}$$

هو الوقت بين ارسال العميل للريكوست وهو مستني الريسبونس

طب اي الفرق بين ال Latency , Response time ؟

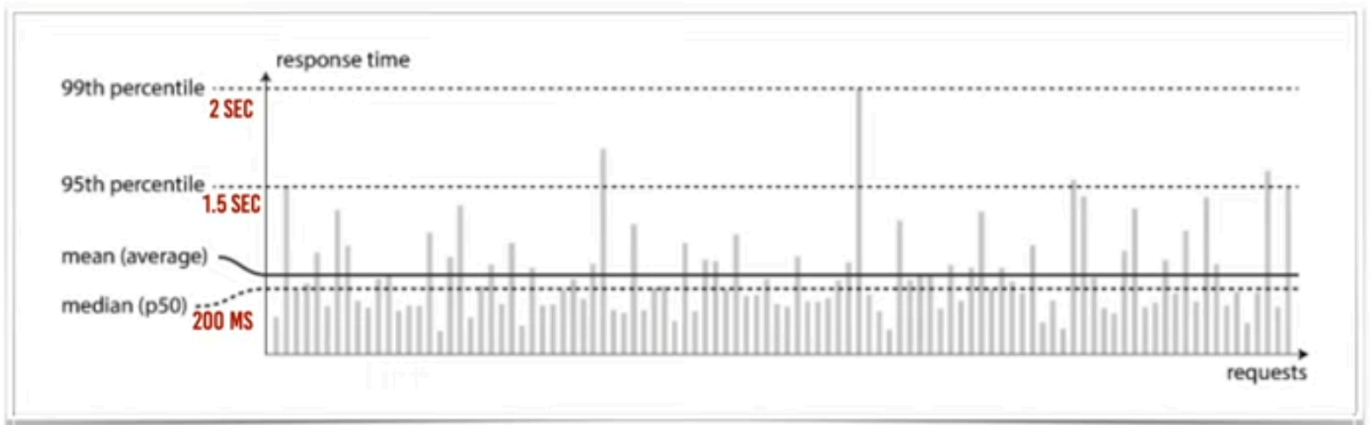
- ال Latency هو عبارة عن المدة الي بينتظهرها الريكوست علشان يتم التعامل معاه
- اما ال Response time هو عبارة عن الوقت الفعلي لمعالجة الطلب وكمان بيشمل التأخير الي فالشبكة والتأخير في الكيوينج وكمان السيرفس برضو
- ال Head of Blocking ودي عبارة عن لو انت ف سوبر ماركت وانت مشتري حاجه معينة وبسيطة خالص وقدامك ناس واقفة فطواير معينة وماسك حاجات كتير هيحاسب عليها، فأنت

مضطر تقف وتستنى الى قدامك يخلص، ف انت دلوقتي معمولك block بسبب الشخص الي قدامك لازم يتعامل معاه الأول

منين اجيبك يا Performance

طب دلوقتي قدرنا نعرف المشاكل

طب ازاى اعرف احسب الاداء بتاعي بإستخدام ال Response time ؟



عندنا ال Percentile وهي النسبة المئوية لل Requests الي بيتم التعامل معاها. ظاهر فالصورة دي ان

Percentile 99th -> 2 sec

Percentile 95th -> 1.5 sec

95 من كل 100 بياخد اقل من ثانية ونص

يعني 5 من اصل 100 بياخدوا ثانية ونص او اكتر!

انت اكيد لو فكرت انك تحاول تخلي النسبة بتاعتك اعلى، فدا هيقابله تكلفة اعلى علشان تقلل وقت الريسبونس تايم، وانت لو فكرت في سناريو انه يخلي كل الريكويستات تاخذ نفس الوقت فدا غالبا مستحيل، بسبب ال TCP , Network Packet loss , random additional latency , retransmission

طب لو انا اعتمدت على المتوسط Mean ؟

الرقم دا اكيد مش هيبقى دقيق 100% لانه مش هيقولك بالضبط كم عدد اليوزرز الي حصلهم مثلا delay

لكن لو اعتمدت على ال Percentile ف انت تقدر تحدد كام عدد من اليوزرز حصلهم Delay بسبب عدد الريكويستات، وطبيعي عدد الريكويستات بتساوي عدد اليوزرز

ال High Percentiles لل Response time او المعروفة باسم tail latencies مهمة زي ما اتكلمنا لأنها بتؤثر بشكل مباشرة على تجربة اليوزرز للخدمة، عندنا مثال لأمازون

امازون عملت دراسة ولقيت مع كل زيادة بقدر 100ms في ال Response time بيقابلها نقص بمقدار 1% في ال Sales وكمال لو حصل تباطؤ بمقدار ثانية واحدة هيققل من مؤشر رضا العميل بنسبة 16%

بسبب انها هتأثر على العملاء الي ممكن يبقى عندهم بطء في الطلبات ودول بيكون عندهم بيانات كتير لو هما عملوا بعدد كبير من المشتريات،

انواع ال Scaling

عندنا نوعين من ال Scaling

- نوع يسمى ب Scaling-Up او ما يعرف ب Vertical Scaling
- عندي مثلا جهاز صغير رامته مثلا 4 ف انا محتاج اكبره لحاجه اكبر وليكن 16 مثلا (اي تطوير في الماشين)
- ونوع يسمى بال Scaling-Out او ما يعرف ب Horizontal Scaling
- عندي ماشين واحدة ف بدل ما اكبرها اوزع اللود على اكتر من ماشين
- توزيع الحمل عبر عدة اجهزة برضو يعرف ب Shared-Nothing Architecture

الوصفة السحرية؟

هو في وصفة سحرية اقدر اعملها بحيث تخلي اي سيستم انا شغال عليه او بعمله يكون Scalable ؟

مكنش حد غلب والله يا صحي

- انت لازم تحدد المشكلة الي بتواجه السيستم بتاعك وبناءا على المشكلة دي انت هتحلها ازاى
- ممكن مثلا الالب بتاعك عدد الريدز أو الريتس فيه كتيرة
- وممكن في ابب ثاني الريسبونس تايم بتاعه كبير فمحتاج تقلله وغيره..
- ف أنت في الأغلب مشكلتك هتبقى ميكس ما بين دول

Maintainability

أو "إزاي متكرهش نفسك بعد سنة"

أحنا اتكلمنا على:

- Reliability → السيستم يفضل شغال حتى لو الدنيا ولعت
- Scalability → السيستم يعرف يستحمل زيادة اللود

طيب الكتاب بيقولك فيه ضلع تالت مهم جدًا في المثلث دا:

Maintainability

أو قابلية الصيانة والتطوير.

الصدمة الأولى

معظم الناس فاكدة إن أغلب تكلفة المشروع بتكون وقت ما بيتبني، بس الحقيقة عكس كدا خالص

أغلب الفلوس والوقت بيتصرفوا بعد ما المشروع يشتغل بالفعل.

يعني بعد ما تعمل Release وتقول خلاص الحمد لله خلصنا من وش ام العميل دا

المعاناة الحقيقية لسه بتبدأ

لأن بعد كدا هيبقى عندك:

- Bugs محتاجة تتصلح
- Failures محتاجة تحقق فيها

• تحديثات أمنية

- Features جديدة

• تغيير Business Requirements

- لمنصات جديدة Migration
- لازم يتسدد Technical Debt

بمعنى تاني

لو مشروعك عاش 10 سنين، غالبًا أول سنة كانت Development والـ 9 سنين الباقيين Maintenance.

ليه الناس بتكره ال Legacy Systems ؟

تعالى نبص على السيناريو الشهير.

تدخل شركة جديدة...

يقولوك:

- تعالى عدل Bug بسيط في السيستم.

تفتح المشروع:

- 700 ألف سطر كود.

- 15 سنة شغال.

- اشتغلوا عليه قبلك 20 Developer.

- مفيش Documentation.

- أسماء Variables غريبة.

تلاقي:

```
var x1 = ProcessData(y2, z3);
```

وتتعد تسأل:

ادي بتعمل اي أصلاً؟ ProcessData

وده اللي بنسميه Legacy System.

الكتاب بيقولك هنا:

كل Legacy System بائس بطريقته الخاصة.

عشان كذا بدل ما نفكر إزاي نصلح الكوارث دي بعدين، نفكر من الأول إزاي منعملهاش.

الثلاث مبادئ لـ Maintainability

الكتاب بيقول أي سيستم محترم لازم يهتم بـ 3 حاجات:

1. Operability

- 2. Simplicity
 - 3. Evolvability
-

أولاً: Operability

أو:

"متقرفش فريق الـ Operations"

مين دول أصلاً؟

دول الناس المسؤولة عن تشغيل السيستم في الإنتاج (Production). سعان الي انت بتجربله لما السيستم بتاعك يخرب

هما بيعملوا اي؟

فريق الـ Operations مسؤول عن حاجات كثير جداً:

- مونتورينج للسيستم: يعني يراقبوا كل من
 - الـ CPU
 - الـ Memory
 - الـ Network
 - الـ Databaseويشوفوا إذا كان فيه مشكلة ولا لا.
- حل الأعطال
 - مثلاً في وقت متأخر كذا السيستم وقع لا قدر الله يعني
 - الناس دي لازم تعرف
 - اي الي وقع بالضبط؟
 - ليه وقع؟
 - يرجع ازاي؟
- التحديثات الأمنية: مثلاً ظهر Security Vulnerability، فكدا لازم يحدثوا كل من
 - الـ OS
 - الـ Database
 - الـ Frameworksقبل ما حد يستغل الثغرة.

- Capacity Planning: يعني بدل ما نستنى السيرفر يقع
 - نتوقع المشكلة قبل ما تحصل، فعلى سبيل المثال
 - لو عدد اليوزرز بيزيد كل شهر 20% ف هنضطر كدا بعد 6 شهور مثلاً هحتاج servers زيادة.
 - المحافظة على معرفة الشركة
 - لازم يكون في توثيق لكل معرفة للشركة دوكيومنتيشن وكل خطوة بتتاخد علشان لو الي شغال على السيستم مشي، منعدهش نلطم
-

يعني اي Good Operability ؟

يعني تخلي حياة الـ Operations Team سهلة.

1- Monitoring محترم

لازم يكون عندك Logs و Metrics واضحة.
بدل:

```
Something went wrong
```

يبقى:

```
Database connection timeout  
OrderId = 12345
```

2- Automation

بدل ما كل Deployment يتم يدويًا.

اعمل:

- CI/CD
- Automated Backups
- Automated Monitoring

كل ما الأتمتة تزيد...الأخطاء البشرية تقل.

3- عدم الاعتماد على جهاز واحد (Scaling-Out)

أسوأ جملة ممكن تسمعها:

السيرفر دا مينفعش يتقفل.

لو جهاز واحد وقع والسيستم كله مات... اكيد يبقى التصميم فيه مشكلة. (كونسبت جديد هنعرف)

4- Documentation

لازم يكون عندك دليل واضح.

مثلاً:

لو عملت:

```
Setting X = True
```

اي اللي هيحصل؟ لو محدش فاهم...

هيبدأ التخمين او بمعنى اصح الهبد هيخرب الدنيا علشان المصريين مش بيعرفوا يسكرتوا (أخوكم من ديروط)

5- سلوك متوقع

أسوأ نوع من السيستمز:

اللي كل يوم يفاجئك بحاجة جديدة.

الكتاب بيقول:

Predictable systems are good systems.

ثانياً: Simplicity

أو بمعنى اصح:

حارب التعقيد قبل ما يقتلك.

كل مشروع بيبدأ كدا:

Users
Orders
Products

جميل وبسيط ولذيذ

بعد سنتين:

Users
Orders
Products
ArchivedOrders
SpecialOrders
TemporaryOrders
BackupOrders
EmergencyOrders

بيحصل اكسباند كذا وتطوير وريفاكتورنج وبتاع

اي أعراض التعقيد؟

الكتاب ذكر شوية أعراض خطيرة جدًا.

• الـ Tight Coupling

- ياه فكرني بالديزاين باترنز (الله يرحمك يا فاكثوري ميثود انتي والاوبسيرفر)
- يعني كل Module ماسك في التاني. لو عدلت حاجة صغيرة هنا... أجزاء تانية تبوظ.

• الـ Tangled Dependencies

- يعني Dependencies متشابكة بطريقة مرعبة.

B يعتمد على A
C يعتمد على B
D يعتمد على C
A يعتمد على D

وحلزونة ياما الحلزونة

- Inconsistent Naming ال
في ملف:

Customer

وفي ملف ثاني:

Client

وفي ملف ثالث:

User

وكلهم نفس الحاجة أصلاً. ارسالك على حل يا عم انت

- ال Hacks
- ودي بتحصل كتير.
- المفروض تصلح المشكلة صح لكن بدل دا بتعمله Patch (بترقعه يعني) سريع.
- بس تيجي بعد سنة: اي يا ابو سيستم مرقع (حتى السيستم رقعتوه)

المشكلة الكبيرة وهي الكومبلكستي

كل ما التعقيد يزداد:

- التطوير يبقى أبطأ
- ال Bugs تزيد
- الفهم يبقى أصعب
- تكلفة الصيانة تزيد

يعني اي Accidental Complexity ؟

من أجمل النقاط في الشابتر.

في نوعين من التعقيد:

- ال Essential Complexity

- تعقيد طبيعي في المشكلة نفسها. مثال:
 - نظام بنك، طبيعي يبقى فيه قواعد وتحويلات وحسابات كثيرة.
 - ال Accidental Complexity: تعقيد أنت اللي صنعته بإيدك.
-

السلاح السري: Abstraction

الكتاب بيعتبر ال Abstraction من أقوى الأسلحة ضد التعقيد.

ايوة يبويا شغلي شغل ال OCP و ال DI و ال SRP بقى

يعني اي Abstraction ؟

اخفيالي الـزنس لوجيك المعقد دا ورا واجهة بسيطة.

مثال

عندنا في ال C# بنكتب:

```
Console.WriteLine("Hello");
```

هل احنا هنا بنتعامل مع:

- CPU Register
- Machine Code
- System Calls

شغل الكومبايلر دا بعيد عني، فقطعا لا.

ال Language خبت كل دا عننا

فايدة ال Abstraction

بدل ما تعيد اختراع العجلة كل مرة.

تعمل Component محترم، وتستخدمه في:

- Project A
- Project B

- Project C

ولو حسنت الـ Component مرة واحدة...
كل المشاريع تستفيد.

ثالثاً: Evolvability

أو:

قابلية النظام للتطور والتغيير.

الكتاب يقولك كذا:
لو فاكـر Requirements مشروعك هتفضل ثابتة للأبد...فأنت متفائل زيادة عن اللزوم
لأن دائماً هيحصل:

- جديدة Features
- جديدة Business Rules
- جدد Users

• قوانين جديدة

- جديدة Platforms
مثال بسيط
بسيط يدعم E-commerce عملت
في البداية عملية الدفعة هتبقى مثلاً كاش بس

بعد سنة:
العميل قالك:

- عايزين Visa.
- عايزين Vodafone Cash.
- عايزين Apple Pay.

لو الكود متصمم صح...التعديل هيكون سهل.

لو متصمم غلط...هتفتح 200 ملف وهتحرق توكـنـز وحلاني بقى
عشان كذا ظهرت أفكار زي:

- Agile
- TDD
- Refactoring

كلها هدفها الأساسي:

| تسهيل التغيير المستقبلي.

نرجع لمثال تويتر مرة ثانية

فاكر مشكلة الـ Timeline؟

كان عندهم حالة اولى وبعدين اكتشفوا إنها مش مناسبة. فانتقلوا على طول للحالة الثانية وبعدين دمجوا الاثنين

لو الـ Architecture بتاعتهم متقفلة ومش قابلة للتغيير... كان التغيير دا هيبقى شبه مستحيل. لكن لأن النظام قابل للتطور... قدروا يغيروا التصميم مع الوقت.

اي العلاقة بين Simplicity و Evolvability

الكتاب ختمها لك ب حاجة حلوة خالص
السيستم البسيط غالبًا:

- أسهل في الفهم
- أسهل في التعديل
- أسهل في الاختبار
- أسهل في التطوير

عشان كذا:

| الـ Simplicity بتؤدي للـ Evolvability
كل ما النظام يبقى أبسط... كل ما تغييره في المستقبل يبقى أسهل.

خلاصة الشاير

لو Reliability بتسأل:

هل السيستم هيشغل؟

ولو Scalability بتسأل:

هل السيستم هيستحمل زيادة اللود؟

ف Maintainability بتسأل:

بعد 5 سنين... هل المهندس اللي هيشغل على السيستم هيشكرني ولا هيدعي عليا؟

ولتحقيق Maintainability لازم تهتم بـ:

- Operability → تشغيل وصيانة سهلة.
- Simplicity → تقليل التعقيد.
- Evolvability → سهولة التعديل والتطوير.

والفكرة الأساسية في الشابتراكه:

الكود مش بيتكتب مرة واحدة ويتساب... الكود الحقيقي بيعيش سنين، فلازم تبنيه بطريقة تخلي المستقبل أرحم شوية.

كل الشكر والتقدير لبشمهندس احمد الإمام على الفيديو الجميل اللذيذ الي هو عامله من ضمن سلسلة فيديوهات اكيد مش اقل جمال من الفيديو دا وانصحكم تسمعوها لو هتبدأوا في الكتاب الجميل جدا دا



صانع محتوى يستهدف ولاد الحلال

Tech بالعربي

أحمد الإمام - Ahmed Elemam

@ahmdelemam • 54.3K subscribers • 226 videos

...more

X and 1 more link

Subscribed

Join

Home Videos Shorts Live Podcasts Courses Playlists Posts

إزاي أحسن ال Soft Skills

Soft Skills

إزاي أحسن ال Soft Skills

Ahmed Elemam - أحمد الإمام • 17K views • Streamed 2 years ago

How to improve my Soft Skills in Arabic with @ahmadalfy @bashmohandes @essamcafe Tech Podcast بالعربي Follow me on Twitter :- <https://twitter.com/ahmdelemam>

2:11:47

For You

دا اول فيديو الي انا سمعته في الفصل دا والبلاي ليست الي بيشرحها
شرح كتاب [Designing Data Intensive Applications](#)

صفحات بشمهندس احمد:

[Ahmed El Emam - LinkedIn](#)

[Ahmed El Emam - X](#)